

Week 6 CoreOps Assignments

MONDAY: GitHub Actions / GitLab CI Pipelines with Terraform & Docker

Objective:

Learn to build a simple CI pipeline that:

- Builds a Docker image
- Runs tests
- Validates Terraform
- Outputs results correctly

SETUP: A sample application

- `mkdir week6-coreops && cd week6-coreops`
- `echo 'print("Hello CoreOps Week 6!")' > app.py`

Create Dockerfile:

```
FROM python:3.9-slim
COPY app.py /app/app.py
CMD ["python", "/app/app.py"]
```

Create Terraform folder: `mkdir infra`

Inside infra/[main.tf](#):

```
terraform { required_version = ">= 1.0" }
```

```
resource "null_resource" "example" { }
```

LAB 1: Create GitHub Actions file

Create .github/workflows/ci.yml: name: CI Pipeline

on: [push]

Jobs:

Build:

runs-on: ubuntu-latest

Steps:

- name: Checkout

uses: actions/checkout@v3

- name: Build Docker image

run: docker build -t coreops-app.

- name: Run container test

run: docker run coreops-app

- name: Setup Terraform

uses: hashicorp/setup-terraform@v2

- name: Terraform Init

run: terraform -chdir=infra init

- name: Terraform Validate

run: terraform -chdir=infra validate

Push to GitHub → observe pipeline.

THE DEVOPS WAR ROOM

WHAT TO OBSERVE

- Docker builds successfully
- Container runs and prints output
- Terraform initializes correctly
- No secrets are hardcoded

Why this matters: This is the foundation of **DevSecOps** automation.

TUESDAY: Generate SBOM + Vulnerability Scanning + Image Signing

Objective:

Use 3 essential DevSecOps tools:

- **SBOM generation** → What is inside your image
- **Scanning** → What vulnerabilities exist
- **Signing** → Prove image authenticity

SETUP: Build your image again

`docker build -t coreops-app:1.0.`

LAB 1: Generate SBOM

Using Syft: `syft coreops-app:1.0 -o json > sbom.json`

Check SBOM file: `cat sbom.json | head`

LAB 2: Scan vulnerabilities (Trivy)

`trivy image coreops-app:1.0`

Expected:

- List of HIGH/CRITICAL vulnerabilities
- Suggestion to update the base image

LAB 3: Sign the image using Cosign

Generate key pair: `cosign generate-key-pair`

Sign image: `cosign sign --key cosign.key coreops-app:1.0`

Verify signature: `cosign verify --key cosign.pub coreops-app:1.0`

WHAT TO OBSERVE

- SBOM lists all installed components
- Trivy reports CVEs
- Signature verification succeeds

Why this matters: Security is **shift-left** → validate **before shipping**.

WEDNESDAY: ArgoCD GitOps: Deploy App + Drift Detection

Objective:

Learn the basics of GitOps:

- App defined in Git
- ArgoCD syncs it
- Manual drift → ArgoCD detects
- Auto-sync fixes it

SETUP: Create a simple manifest repo

Create manifests/deployment.yaml:

apiVersion: apps/v1

kind: Deployment

Metadata:

name: gitops-app

Spec:

replicas: 2

Selector:

matchLabels:

app: gitops-app

Template:

Metadata:

Labels:

app: gitops-app

Spec:

Containers:

- name: app

image: nginx

Push to GitHub.

LAB 1: Create ArgoCD App

In ArgoCD UI or via CLI: `argocd app create gitops-app \ --repo`

`https://github.com/<your-user>/<your-repo> \ --path manifests \--dest-server`

`https://kubernetes.default.svc \--dest-namespace default`

Sync: `argocd app sync gitops-app`

LAB 2: Induce Drift

kubectl scale deploy gitops-app --replicas=5

Check ArgoCD:

- Shows **OutOfSync**
- Shows what changed

LAB 3: Enable Auto-Sync

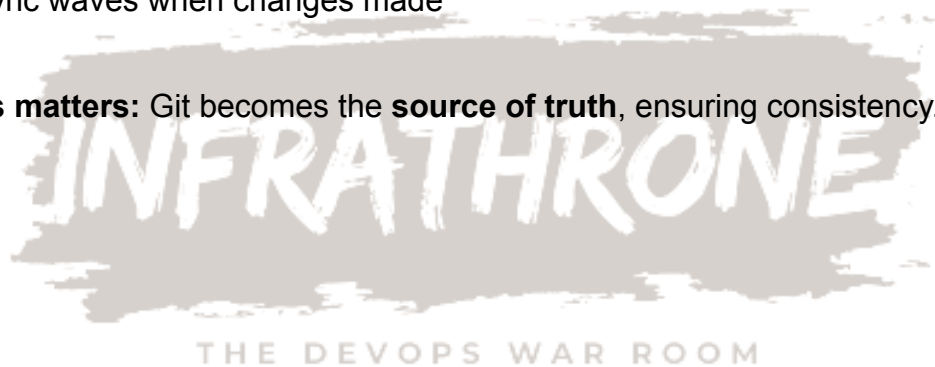
argocd app set gitops-app --sync-policy automated

Observe auto-correction: kubectl get deploy gitops-app

WHAT TO OBSERVE

- Drift detection
- Rollback to Git state
- Sync waves when changes made

Why this matters: Git becomes the **source of truth**, ensuring consistency.



THURSDAY: Internal Developer Platform Basics (Golden Paths)

Objective:

Learn to build a simple internal platform workflow:

- Golden path app template
- Namespace provisioning
- RBAC automation

SETUP: Create a platform folder

```
mkdir platform && cd platform
```

LAB 1: Golden Path Template (simple app)

Create template.yaml:

```
apiVersion: v1
kind: Namespace
metadata: name: {{appName}}
apiVersion: apps/v1
kind: Deployment
metadata: name: {{appName}}
namespace: {{appName}}
spec: replicas: 1
Selector:
matchLabels:
app: {{appName}}
template: metadata:
Labels:
app: {{appName}}
Spec:
Containers:
- name: app
image: nginx
```

LAB 2: Self-Service Namespace Provisioning Script

Create provision.sh:

```
#!/bin/bash
name=$1
kubectl create ns $name
kubectl apply -f template.yaml -n $name --dry-run=client \ | sed
"s/{{appName}}/$name/g" | kubectl apply -f -Run:bash provision.sh demo-app
```

LAB 3: Add RBAC Guardrails

```
kubectl create role developer \ --verb=get,list,watch \--resource=pods,deployments \
-n demo-app
```

```
kubectl create rolebinding developer-binding \ --role=developer
\--user=<your-user-email> \ -n demo-app
```

WHAT TO OBSERVE

- Namespace created
- Deployment deployed
- RBAC applied correctly

Why this matters: Platforms remove friction and prevent misconfigurations.

THE DEVOPS WAR ROOM

FRIDAY: Backstage: Catalog + Templates + Scorecards

Objective: Learn how Backstage powers internal developer portals.

SETUP: Run Backstage container

```
npx @backstage/create-app  
cd my-backstage-app  
yarn dev  
Backstage opens at localhost:3000.
```

LAB 1: Register a Service

Create catalog-info.yaml:

```
apiVersion: backstage.io/v1alpha1  
kind: Component  
Metadata:  
name: coreops-service
```

Register via Backstage UI.

LAB 2: Create a Template (Scaffolder)

Inside /templates/basic-service/template.yaml:apiVersion:

```
scaffolder.backstage.io/v1beta3
```

```
kind: Template
```

```
Metadata:
```

```
name: basic-service
```

```
Spec:
```

```
owner: platform-team
```

```
Steps:
```

```
- id: scaffold
```

```
  action: fetch:template
```

```
  Input:
```

```
url: ./skeleton
```


LAB 3: Scorecards

Install: `yarn add @backstage/plugin-scorecards`

Create `scorecards.yaml`:

Checks:

name: has-ci-pipeline

description: CI pipeline should exist

type: file

file: pattern: "^github/workflows/.*/.*\\.yaml\$"

WHAT TO OBSERVE

- Service appears in catalog
- Template generates files
- Scorecard evaluates best practices

Why this matters

Backstage centralizes:

- service discovery
- documentation
- onboarding
- templates
- quality checks



This is the foundation of any **internal developer platform**.